國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab 05

姓名： 紀宥丞

學號： 114310221

Lab05-Ex1.
Exercise: Shuffling and Dealing Cards (Use Stack)
Code

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <random>
using namespace std;

class Card {
public:
    string colors; // 儲存撲克牌的花色
    string rank;    // 儲存撲克牌的數值

    Card(string s, string r) : colors(s), rank(r) {}

    void display() const {
        cout << rank << " of " << colors << endl;
    }
};
// 實作 Stack
class Stack {
private:
    vector<Card> stack; // 表示 stack 是一個能存放 Card 類別物件的 vector

public:
    // 將一張牌放入堆疊頂端
    void push(const Card& card) {
        // 利用 vector 的 push_back() 函式，將傳入的卡牌放置到陣列的最後面（即堆疊的頂端）
        stack.push_back(card);
    }

    // 從堆疊頂端取出一張牌
    Card pop() {
        if (!isEmpty()) {
            Card topCard = stack.back(); // 先用 back() 取得堆疊最上方的卡牌資料
```

```
            stack.pop_back();                // 利用 pop_back() 將最上方的卡牌從堆疊中移除
            return topCard;                  // 回傳被取出的卡牌
        } else {
            // 防呆機制：如果堆疊是空的，印出提示並回傳一張空卡牌
            cout << "Stack 是空的！" << endl;
            return Card("", "");
        }
    }

    // 檢查堆疊是否為空
    bool isEmpty() const {
        // 若 vector 內沒有元素則 empty() 會回傳 true，否則回傳 false
        return stack.empty();
    }
};
// 代表一副撲克牌的類別
class Deck {
private:
    vector<Card> cards; // 存放未洗牌的撲克牌
    Stack shuffledDeck; // 存放洗過的牌，用實作的 stack 來管理

public:
    // 建立 constructor 來初始化物件
    Deck() {
        string colors[] = { "Hearts", "Diamonds", "Clubs", "Spades" }; // 四種花色
        string ranks[] = { "2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K", "A" }; // 13 種點數
        // 利用迴圈將 52 張牌加入 cards 這個 vector 裡面
        for (int i = 0; i < 4; i++) { // 四種花色
            for (int j = 0; j < 13; j++) { // 13 種點數
                cards.push_back(Card(colors[i], ranks[j]));
            }
        }
    }
//洗牌
void shuffleDeck() {
  // 設定標準的亂數產生器，作為洗牌的隨機亂數種子
```

```
            random_device rd;
            mt19937 g(rd());
            shuffle(cards.begin(), cards.end(), g);
   // 1.將 cards 陣列裡的卡牌使用 std::shuffle 打亂順序

            for (int i = 0; i < cards.size(); i++) {
              // 2.將洗好的卡牌一張一張放入 (push) 到 shuffledDeck 堆疊中
                shuffledDeck.push(cards[i]);
            }
      }
//發牌
void drawAllCards(){
   // 利用 while 迴圈，只要 shuffledDeck 堆疊不是空的就繼續發牌
            while (!shuffledDeck.isEmpty()) {
                Card drawnCard = shuffledDeck.pop(); // 3.從堆疊的頂端取出一張卡牌
(pop)
                drawnCard.display();                    // 呼叫 Card 類別內的函式將卡
牌資訊顯示在螢幕上
            }
      }
};

int main() {
      Deck deck;              // 建立 deck 產生 52 張撲克牌
      deck.shuffleDeck(); // 進行洗牌並放入堆疊

      cout << "Shuffled deck:" << endl;
      deck.drawAllCards(); // 依序取出堆疊內的牌並顯示

      return 0;
}
```